

Технические материалы для внедрения развертывания mock-django

Шаблоны .gitlab-ci.yml, docker-compose.prod.yml, .env, deploy.sh и правок Django settings

Назначение пакета

Этот PDF содержит проектные шаблоны материалов, которые нужно добавить в репозиторий mock-django или использовать как основу для MR: структура файлов, .gitlab-ci.yml, docker-compose.prod.yml, .env.example, deploy.sh, изменения settings.py и README_DEPLOY. Шаблоны рассчитаны на перенос в GitLab CI/CD и Docker Compose-развертывание на сервере ЦБС.

Предлагаемая структура репозитория

```
mock-django/
.gitlab-ci.yml           # pipeline GitLab CI/CD
Dockerfile              # production image; доработать non-root/health deps
docker-compose.yml      # локальный запуск, можно оставить
docker-compose.prod.yml # runtime на staging/prod сервере
.env.example            # пример без секретов
scripts/
  deploy.sh             # атомарный deploy через SSH/Compose
  smoke.sh              # smoke tests после деплоя
app/
  manage.py
  config/settings.py     # env-based production settings
README_DEPLOY.md        # инструкция для администратора и релиз-инженера
```

Шаблон .gitlab-ci.yml

Pipeline воспроизводит переносимую схему: проверить Django, собрать контейнер, опубликовать образ в GitLab Container Registry, развернуть на staging автоматически и production вручную.

```
stages:
  - lint
  - test
  - build
  - deploy

variables:
  IMAGE_TAG: "${CI_REGISTRY_IMAGE}:${CI_COMMIT_SHORT_SHA}"
  IMAGE_LATEST: "${CI_REGISTRY_IMAGE}:latest"
  DOCKER_TLS_CERTDIR: "/certs"

workflow:
  rules:
    - if: '$CI_COMMIT_BRANCH'
    - if: '$CI_COMMIT_TAG'

lint:django-check:
  stage: lint
  image: python:3.11-alpine
  before_script:
    - apk add --no-cache gcc musl-dev postgresql-dev
    - pip install --no-cache-dir -r requirements.txt
  script:
    - cd app
    - python manage.py check --deploy
  rules:
    - if: '$CI_COMMIT_BRANCH'

test:django:
  stage: test
  image: python:3.11-alpine
  services:
    - name: postgres:15-alpine
      alias: db
  variables:
    POSTGRES_DB: testdb
    POSTGRES_USER: testuser
    POSTGRES_PASSWORD: testpass
    POSTGRES_HOST: db
    SECRET_KEY: test-secret-key-change-me
    ALLOWED_HOSTS: localhost,127.0.0.1
  before_script:
    - apk add --no-cache gcc musl-dev postgresql-dev
    - pip install --no-cache-dir -r requirements.txt
```

```

script:
  - cd app
  - python manage.py test --noinput
rules:
  - if: '$CI_COMMIT_BRANCH'

build:image:
  stage: build
  image: docker:27
  services:
    - docker:27-dind
  before_script:
    - docker login -u "$CI_REGISTRY_USER" -p "$CI_REGISTRY_PASSWORD" "$CI_REGISTRY"
  script:
    - docker build --pull -t "$IMAGE_TAG" -t "$IMAGE_LATEST" .
    - docker push "$IMAGE_TAG"
    - docker push "$IMAGE_LATEST"
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
    - if: '$CI_COMMIT_TAG'

deploy:staging:
  stage: deploy
  image: alpine:3.20
  before_script:
    - apk add --no-cache openssh-client bash
    - eval $(ssh-agent -s)
    - echo "$STAGING_SSH_PRIVATE_KEY" | tr -d '
' | ssh-add -
    - mkdir -p ~/.ssh && chmod 700 ~/.ssh
    - ssh-keyscan -H "$STAGING_HOST" >> ~/.ssh/known_hosts
  script:
    - ./scripts/deploy.sh staging "$IMAGE_TAG"
  environment:
    name: staging
    url: "https://$STAGING_DOMAIN"
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'

deploy:production:
  stage: deploy
  image: alpine:3.20
  before_script:
    - apk add --no-cache openssh-client bash
    - eval $(ssh-agent -s)
    - echo "$PROD_SSH_PRIVATE_KEY" | tr -d '
' | ssh-add -
    - mkdir -p ~/.ssh && chmod 700 ~/.ssh
    - ssh-keyscan -H "$PROD_HOST" >> ~/.ssh/known_hosts
  script:
    - ./scripts/deploy.sh production "$IMAGE_TAG"
  environment:
    name: production
    url: "https://$PROD_DOMAIN"
  when: manual
  allow_failure: false
  rules:
    - if: '$CI_COMMIT_TAG'

```

Шаблон docker-compose.prod.yml

```

services:
  web:
    image: ${APP_IMAGE}
    container_name: mock-django-web
    command: >
      sh -c "python manage.py migrate --noinput &&
        python manage.py collectstatic --noinput || true &&
        gunicorn config.wsgi:application --bind 0.0.0.0:8000 --workers 3"
    env_file:
      - .env
    expose:
      - "8000"
    environment:
      VIRTUAL_HOST: ${APP_DOMAIN}
      POSTGRES_HOST: db
    depends_on:
      db:
        condition: service_healthy
    healthcheck:
      test: ["CMD-SHELL", "wget -qO- http://localhost:8000/health/ >/dev/null || exit 1"]
      interval: 30s
      timeout: 5s
      retries: 5
    restart: unless-stopped
    networks:
      - mock-network

  db:
    image: postgres:15-alpine

```

```

container_name: mock-django-db
env_file:
  - .env
volumes:
  - postgres_data:/var/lib/postgresql/data
  - ./backups:/backups
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
  interval: 10s
  timeout: 5s
  retries: 10
restart: unless-stopped
networks:
  - mock-network

nginx-proxy:
  image: jwilder/nginx-proxy:alpine
  container_name: mock-django-nginx-proxy
  ports:
    - "80:80"
    - "443:443"
  volumes:
    - /var/run/docker.sock:/tmp/docker.sock:ro
    - ./certs:/etc/nginx/certs:ro
  restart: unless-stopped
  networks:
    - mock-network

volumes:
  postgres_data:

networks:
  mock-network:
    driver: bridge

```

Шаблон .env.example

```

# Do not use real secrets in repository. Copy to /opt/mock-django/.env on server.
APP_IMAGE=registry.example.org/group/mock-django:latest
APP_DOMAIN=mock-django.cbs-petrograd.example.org
DJANGO_DEBUG=False
SECRET_KEY=change-me-use-gitlab-variable-or-server-secret
ALLOWED_HOSTS=mock-django.cbs-petrograd.example.org
CSRF_TRUSTED_ORIGINS=https://mock-django.cbs-petrograd.example.org
CSRF_COOKIE_SECURE=True
SESSION_COOKIE_SECURE=True
POSTGRES_DB=mockdb
POSTGRES_USER=mockuser
POSTGRES_PASSWORD=change-me-long-random-value
POSTGRES_HOST=db
POSTGRES_PORT=5432

```

Переменные GitLab CI/CD

Переменная	Где используется	Тип/защита
STAGING_HOST, STAGING_DOMAIN	deploy:staging	Protected не обязательно, Masked для чувствительных значений.
STAGING_SSH_PRIVATE_KEY	SSH-деплой staging	Masked, Protected, File/Variable; доступ только Maintainer.
PROD_HOST, PROD_DOMAIN	deploy:production	Protected.
PROD_SSH_PRIVATE_KEY	SSH-деплой production	Masked, Protected, лучше отдельный deploy key без интерактивного shell.
CI_REGISTRY, CI_REGISTRY_IMAGE, CI_REGISTRY_USER, CI_REGISTRY_PASSWORD	Build/push image	Predefined GitLab variables, не переопределять.
SECRET_KEY, POSTGRES_PASSWORD	Серверный .env или CI-generated .env	Не хранить в репо; Protected/Masked либо secret file на сервере.

Шаблон scripts/deploy.sh

```

#!/usr/bin/env bash
set -euo pipefail

ENVIRONMENT="${1:?environment required: staging|production}"
APP_IMAGE="${2:?docker image tag required}"

case "$ENVIRONMENT" in
  staging)
    DEPLOY_HOST="$STAGING_HOST"
    DEPLOY_USER="${STAGING_USER:-deploy}"
    DEPLOY_DIR="${STAGING_DEPLOY_DIR:-/opt/mock-django}"

```

```

;;
production)
DEPLOY_HOST="$PROD_HOST"
DEPLOY_USER="${PROD_USER:-deploy}"
DEPLOY_DIR="${PROD_DEPLOY_DIR:-/opt/mock-django}"
;;
*)
echo "Unknown environment: $ENVIRONMENT" >&2
exit 2
;;
esac

ssh "$DEPLOY_USER@$DEPLOY_HOST" "mkdir -p '$DEPLOY_DIR'"
scp docker-compose.prod.yml "$DEPLOY_USER@$DEPLOY_HOST:$DEPLOY_DIR/docker-compose.yml"

ssh "$DEPLOY_USER@$DEPLOY_HOST" "cd '$DEPLOY_DIR' && grep -q '^APP_IMAGE=' .env && sed -i
's#^APP_IMAGE=.*APP_IMAGE=$APP_IMAGE#' .env || echo 'APP_IMAGE=$APP_IMAGE' >> .env && docker
compose pull && docker compose up -d --remove-orphans && docker compose ps && sleep 8
&& docker compose exec -T web wget -qO- http://localhost:8000/health/"

```

Рекомендуемая правка app/config/settings.py

```

import os
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent

SECRET_KEY = os.environ["SECRET_KEY"]
DEBUG = os.getenv("DJANGO_DEBUG", "False").lower() == "true"
ALLOWED_HOSTS = [h.strip() for h in os.getenv("ALLOWED_HOSTS", "").split(",") if h.strip()]
CSRF_TRUSTED_ORIGINS = [
    h.strip() for h in os.getenv("CSRF_TRUSTED_ORIGINS", "").split(",") if h.strip()
]
CSRF_COOKIE_SECURE = os.getenv("CSRF_COOKIE_SECURE", "True").lower() == "true"
SESSION_COOKIE_SECURE = os.getenv("SESSION_COOKIE_SECURE", "True").lower() == "true"
SECURE_PROXY_SSL_HEADER = ("HTTP_X_FORWARDED_PROTO", "https")

INSTALLED_APPS = [
    "django.contrib.contenttypes",
    "django.contrib.staticfiles",
    "core",
]

MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
]

ROOT_URLCONF = "config.urls"
WSGI_APPLICATION = "config.wsgi.application"

DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": os.getenv("POSTGRES_DB"),
        "USER": os.getenv("POSTGRES_USER"),
        "PASSWORD": os.getenv("POSTGRES_PASSWORD"),
        "HOST": os.getenv("POSTGRES_HOST", "db"),
        "PORT": int(os.getenv("POSTGRES_PORT", "5432")),
        "CONN_MAX_AGE": int(os.getenv("DB_CONN_MAX_AGE", "60")),
    }
}

STATIC_URL = "/static/"
STATIC_ROOT = BASE_DIR / "staticfiles"
DEFAULT_AUTO_FIELD = "django.db.models.BigAutoField"

```

README_DEPLOY.md - содержание

- Назначение сервиса, домен, окружения, ответственные лица.
- Локальный запуск: `docker compose up -d --build`; staging/prod запуск только через CI/CD.
- Список переменных окружения и где они задаются: GitLab variables или `/opt/mock-django/.env`.
- Релиз: создать tag, запустить manual production deploy, проверить `/` и `/health/`.
- Rollback: выбрать предыдущий image tag, обновить APP_IMAGE, `docker compose pull && up -d`.
- Backup: `pg_dump`, проверка восстановления, retention, расположение файлов.
- Диагностика: `docker compose ps/logs/exec`, статус healthcheck, типовые ошибки.

Источники и проверенные факты

Дата подготовки: 20 июня 2026 г. Источники использованы для инвентаризации репозиторий и обоснования технических решений.

- GitLab: <https://gitlab.herzen.spb.ru/1148905/prog-6-ml-service> - публичная карточка проекта: Project ID 399, 29 commits, 2 branches, наличие CI/CD configuration, Container Registry, файл .gitlab-ci.yml 2.86 KB; содержимое .gitlab-ci.yml не было доступно как текст без авторизации/рендера.
- GitHub: <https://github.com/schudnaya/mock-django> - публичный репозиторий mock-django, 10 commits, файлы app, Dockerfile, README.md, docker-compose.yml, requirements.txt; README содержит локальный запуск через docker compose up -d --build и переход на <http://localhost:8080>.
- Dockerfile mock-django: Python 3.11 Alpine, установка системных пакетов, pip install -r requirements.txt, запуск Gunicorn config.wsgi:application на 0.0.0.0:8000.
- docker-compose.yml mock-django: сервисы web, db(PostgreSQL 15 Alpine), nginx-proxy; порт хоста 8080 -> 80; переменные POSTGRES_* заданы в compose.
- Django settings.py mock-django: SECRET_KEY задан в коде, DEBUG=False, ALLOWED_HOSTS=["*"], подключение к PostgreSQL через переменные окружения; URL /health/ выполняет SELECT 1 к БД.
- GitLab Docs: .gitlab-ci.yml описывает jobs pipeline; predefined variables доступны в каждом pipeline; CI/CD variables применяются для переиспользования значений и предотвращения hard-code секретов; GitLab CI/CD может строить, тестировать и push Docker images.
- Docker Docs: Compose создает сервисы в порядке зависимостей; healthcheck/depends_on с condition: service_healthy позволяют ждать готовности зависимого сервиса.
- Django Docs: deployment checklist рекомендует manage.py check --deploy, секретный SECRET_KEY вне source control, DEBUG=False в production, конкретные ALLOWED_HOSTS, STATIC_ROOT/collectstatic, HTTPS и secure cookies.